Project Handover & Build Guide

PROJECT 2 OF 2

Real-Time Stock Market Forecaster

What you need to know • What to discuss • How to build it step by step

Mehmet Tanil Kaplan • Student ID: T0429362 • Nottingham Trent University • Supervisor: Williams Magdalena

1. What Is This Project?

This is a full-stack web application that fetches real-world stock market data and uses machine learning to predict the next day's closing price. The defining feature is a direct head-to-head comparison between two models: a simple baseline model and a deep-learning LSTM network.

Layer	Analogy	What it actually does
React.js Frontend	The trading screen	The website users see — stock selector, historical price chart, baseline vs LSTM predictions, error metrics
FastAPI Backend	The trading engine	Receives requests, loads both trained models, fetches latest data, runs predictions, returns JSON
ML Models (.pkl / .h5)	The algorithms	Baseline (Linear Regression or Random Forest) vs LSTM neural network — both saved after training
Yahoo Finance / Alpha Vantage	The data feed	Live and historical OHLCV stock price data (open, high, low, close, volume) via free public APIs

1.1 The Core Idea

A user types a stock ticker (e.g. AAPL, TSLA, MSFT), clicks predict, and sees a chart of historical closing prices with two prediction lines — one from the baseline model and one from the LSTM. The app shows error metrics (MSE and RMSE) for each model so users can see which performs better. A disclaimer banner is always visible.

> **Educational goal — not a trading tool:**
> This system exists to demonstrate that LSTM deep-learning models outperform simple regression on financial time-series data. It is not intended to guide real financial decisions. Disclaimers must appear on every page of the application.

1.2 Key Numbers to Remember

Item	Detail
Accuracy target	LSTM must achieve ≥ 10% lower RMSE than the baseline across all 5 stock tickers

Item	Detail
Prediction output	Next day’s closing price (short-term, single-step forecast)
Input window size	50 consecutive days of closing prices used to predict day 51
Data split	70% training / 15% validation / 15% testing — chronological order, never shuffled
Stock tickers to support	At least 5: recommended AAPL, TSLA, MSFT, GOOGL, AMZN
API response target	Under 1 second per /predict call
Timeline	12 weeks with week-by-week milestones
Ethical requirement	Prominent disclaimer on every page: "For educational purposes only. Not financial advice."

2. What Your Friend Needs to Know

2.1 The Technology Stack

Tool / Library	Purpose	Why it was chosen
Python (pandas, numpy, scikit-learn)	Data processing, baseline model training	Industry-standard; baseline models (Linear Regression, Random Forest) are in scikit-learn
TensorFlow / Keras	LSTM model design and training	Most widely used deep learning framework; Sequential API keeps LSTM code clean and readable
yfinance	Fetching Yahoo Finance stock data	No API key needed; pip install yfinance; most reliable free source for historical price data
Alpha Vantage API	Alternative data source	Free API key at alphavantage.co; more structured but limited to 25 requests/day on free tier
joblib	Baseline model serialisation	Saves the trained sklearn model as model_baseline.pkl
FastAPI + uvicorn	Backend REST API	High-performance Python framework; auto-generates /docs API explorer
React.js + Chart.js	Frontend dashboard	Responsive interactive charts; react-chartjs-2 wraps Chart.js natively for React
Axios	HTTP client in React	Handles all API calls from the frontend to the FastAPI backend
GitHub	Version control	Track all changes; provides rollback if something breaks
Render / Heroku	Cloud deployment	Free platforms hosting Python Web Services and React Static Sites

2.2 Understanding the Two Models

This is the most technically complex part of the project. Your friend needs to understand both models clearly.

The Baseline Model

A simple machine learning model used as a comparison benchmark. Either Linear Regression or Random Forest Regressor from scikit-learn. It takes the 50-day price window as a flat feature vector and predicts the next price. It does not understand sequences or time order — that is its limitation. Save as model_baseline.pkl using joblib.

The LSTM Model (Long Short-Term Memory)

An LSTM is a type of Recurrent Neural Network (RNN) designed specifically for sequences. Standard RNNs suffer from the vanishing gradient problem — they forget information from many steps ago. LSTMs solve this with three internal gates:

- **Input gate:** decides how much new information to store
- **Forget gate:** decides how much old information to discard
- **Output gate:** decides what to pass to the next time step

This gating mechanism lets the LSTM remember patterns across all 50 days of input, not just the last few. That is why it outperforms the baseline on stock price data.

LSTM architecture to implement:

```
from tensorflow.keras.models import Sequential from tensorflow.keras.layers import LSTM,
Dense, Dropout  model = Sequential([     LSTM(64, return_sequences=True,
input_shape=(50, 1)),     Dropout(0.2),     LSTM(32, return_sequences=False),
Dropout(0.2),     Dense(1)  # predicts next day closing price ])
model.compile(optimizer='adam', loss='mse') model.fit(X_train, y_train, epochs=50,
batch_size=32,          validation_data=(X_val, y_val))
```

2.3 The Data Flow

Understanding how data moves through the system is essential before writing any code:

1. **Fetch:** Download historical daily OHLCV data for each ticker using yfinance. Save as CSV files immediately.
2. **Select:** Extract only the Close column for model training.
3. **Scale:** Apply Min-Max Scaling to normalise values between 0 and 1. Save the scaler object — it is needed again at prediction time.
4. **Window:** Create 50-day sliding windows. Each sample is 50 consecutive Close prices; the label is price on day 51.
5. **Split:** Divide chronologically: first 70% for training, next 15% for validation, final 15% for testing. Never shuffle.
6. **Train:** Fit both models on training data. Monitor val_loss to detect overfitting.
7. **Evaluate:** Calculate MSE and RMSE on the test set for both models. Confirm LSTM improves baseline by ≥ 10%.
8. **Save:** model_baseline.pkl (joblib), model_lstm.h5 (Keras), scaler.pkl (joblib).
9. **Serve:** FastAPI loads all three files at startup. On each /predict call, it fetches the latest 50 days, scales, reshapes, predicts, inverse-transforms, and returns JSON.

Critical: always save the scaler

The Min-Max Scaler must be fitted on training data and saved: joblib.dump(scaler, 'models/scaler.pkl'). At prediction time, new data must be scaled with the exact same scaler. Using a freshly fitted scaler with different min/max values will produce completely wrong predictions. This is one of the most common mistakes in ML deployment.

2.4 The API Endpoints

- **GET /predict?ticker=AAPL** — returns baseline prediction, LSTM prediction, and a confidence note as JSON

- **GET /history?ticker=AAPL** — returns the last 90 days of historical closing prices for charting
- **GET /metrics?ticker=AAPL** — returns MSE and RMSE for both models on the test set
- **GET /tickers** — returns the list of supported stock tickers

CORS warning — add this on day one:

When React (port 3000) calls FastAPI (port 8000), the browser blocks it unless CORS is configured. Add to main.py immediately: from fastapi.middleware.cors import CORSMiddleware app.add_middleware(CORSMiddleware, allow_origins=['*'], allow_methods=['*'], allow_headers=['*'])

2.5 The API Rate Limit Strategy

Both Yahoo Finance and Alpha Vantage have usage limits. The correct strategy is to download and cache all historical data as CSV files once, then read from those files during development and at prediction time.

Recommended data caching script (fetch_data.py):

import yfinance as yf tickers = ['AAPL', 'TSLA', 'MSFT', 'GOOGL', 'AMZN'] for ticker in tickers: df = yf.download(ticker, start='2015-01-01', end='2024-12-31') df.to_csv(f'data/{ticker}.csv') print(f'Saved {ticker}.csv — {len(df)} rows') # Run this ONCE. Read from CSV files after that.

For near-real-time prediction, the API only needs to fetch the single most recent day's price — not re-download years of history on every call.

3. What to Discuss with Your Friend

3.1 Skill Check

This project requires Python ML skills, deep learning experience, FastAPI knowledge, and React ability. Check all four honestly before agreeing.

Skill	Level needed	Question to ask
Python (pandas, numpy, scikit-learn)	Essential	Have you cleaned time-series data in pandas before?
TensorFlow / Keras (LSTM)	Needed — willingness to learn is OK	Have you built a neural network before, even a simple one?
FastAPI or Flask	Needed	Have you built and tested a REST API in Python before?
React.js + Chart.js	Mid-level frontend	Are you comfortable with JavaScript and React?
Financial data and APIs	No prior experience needed	Are you comfortable reading API docs and handling JSON?
Git / GitHub	Essential from day one	Do you use Git and GitHub regularly?

If they have never used Keras before:

The Keras Sequential API is high-level and beginner-friendly. Two focused days following the official time-series tutorial at keras.io is usually enough to understand the basics. The LSTM architecture in Section 2.2 can be copied directly as a starting point.

3.2 Key Topics to Agree On Before Work Starts

The 10% RMSE improvement target

This is the measurable success condition for the whole project. The LSTM must achieve at least 10% lower RMSE than the baseline across all 5 tickers. If it does not, your friend needs to tune hyperparameters (more LSTM units, more epochs, different dropout rate, different window size) and try again. Agree that results will be reported honestly regardless of outcome — documenting why a target was hard to hit is acceptable and expected.

The 5 stock tickers

Agree on exactly which tickers to use before any data is downloaded. The recommended set is AAPL, TSLA, MSFT, GOOGL, AMZN. These are large-cap US stocks with 10+ years of daily data available through yfinance with no gaps. Do not choose tickers with short history, frequent trading halts, or thin volume — the model will struggle to find patterns.

Never shuffle the data

This needs to be an explicit agreement. Stock data must stay in chronological order. Using scikit-learn's train_test_split with shuffle=True will let the model see future prices during training, producing artificially perfect scores that collapse on real unseen data. Agree to always split by index position.

The disclaimer requirement

A disclaimer reading "For educational purposes only. This is not financial advice." must appear on every page of the React dashboard. This is a stated ethical requirement in the project brief and cannot be removed to make the app look more polished. Agree on this upfront.

4. How to Build It — Step by Step

Step 1 — Set Up the Environment

10. Create a project folder and run: git init, then push to GitHub
11. Create a Python virtual environment: python -m venv venv
12. Activate: source venv/bin/activate (Mac/Linux) or venv\Scripts\activate (Windows)
13. Install Python libraries: pip install pandas numpy scikit-learn tensorflow keras fastapi uvicorn yfinance joblib matplotlib jupyter
14. Scaffold the React app: npx create-react-app frontend
15. Inside frontend/: npm install chart.js react-chartjs-2 axios

Recommended folder structure:

```
project/ ├── data/ │   ├── AAPL.csv      ← cached historical data per ticker │   ├── TSLA.csv │   └── ... ├── notebooks/       ← Jupyter experiments ├── models/ │   ├── model_baseline.pkl │   ├── model_lstm.h5 │   └── scaler.pkl       ← MUST be saved alongside models ├── backend/ │   ├── main.py          ← FastAPI app │   ├── predict.py ← prediction logic │   └── preprocess.py     ← scaling and windowing helpers ├── frontend/ ← React app └── README.md
```

Step 2 — Fetch and Cache Data

Run fetch_data.py once (see Section 2.5). Save one CSV per ticker. This is the data source for all subsequent steps. Never re-download during API calls.

Do not download on every API call:

If the FastAPI /predict endpoint calls yfinance on every request, it will hit rate limits within minutes and the app will break. Always read from the cached CSV files for historical data. Only call yfinance for the single most recent day's price if needed.

Step 3 — Preprocess and Create Windows

Write a preprocessing script (preprocess.py) that the training scripts and FastAPI backend both import:

16. Load CSV: df = pd.read_csv('data/AAPL.csv', index_col='Date', parse_dates=True)
17. Extract Close column: prices = df[['Close']].values
18. Fit Min-Max Scaler on training data only: scaler = MinMaxScaler(); scaler.fit(train_prices)
19. Scale all data: scaled = scaler.transform(prices)
20. Create 50-day windows using a loop: for i in range(50, len(scaled)): X.append(scaled[i-50:i]), y.append(scaled[i])
21. Split chronologically: train 70%, validation 15%, test 15% by index

22. Save the fitted scaler: joblib.dump(scaler, 'models/scaler.pkl')

Never fit the scaler on all data:
Fitting the scaler on the full dataset (train + validation + test) causes data leakage — the model indirectly sees test data statistics during training. Fit the scaler on training data only, then use that same fitted scaler to transform validation and test data.

Step 4 — Train the Baseline Model

Do this in Jupyter Notebook first to check scores, then write a clean train_baseline.py script.

23. Reshape X: for sklearn, flatten each 50-day window into a 1D feature vector: X_flat = X.reshape(len(X), -1)
24. Train: model = RandomForestRegressor(n_estimators=100); model.fit(X_train_flat, y_train)
25. Predict on test set: y_pred = model.predict(X_test_flat)
26. Inverse-transform predictions back to original price scale: y_pred_actual = scaler.inverse_transform(y_pred.reshape(-1,1))
27. Calculate RMSE: rmse = np.sqrt(mean_squared_error(y_test_actual, y_pred_actual))
28. Record the RMSE. This is the benchmark the LSTM must beat by 10%.
29. Save: joblib.dump(model, 'models/model_baseline.pkl')

Step 5 — Train the LSTM Model

Do this in Jupyter Notebook. Reshape data for LSTM input: (samples, timesteps, features) where features=1.

30. Reshape: X_lstm = X.reshape(X.shape[0], X.shape[1], 1)
31. Build the model using the architecture from Section 2.2
32. Add early stopping to prevent overfitting: EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
33. Train: model.fit(X_train, y_train, epochs=50, batch_size=32, validation_data=(X_val, y_val), callbacks=[early_stopping])
34. Predict on test set and inverse-transform back to original price scale
35. Calculate RMSE. Confirm it is ≥ 10% lower than the baseline RMSE.
36. Save: model.save('models/model_lstm.h5')

If the LSTM does not hit 10% improvement, try these in order:
1. Increase LSTM units: try 128 units in the first layer 2. Train for more epochs: try 100 with early stopping patience=15 3. Reduce dropout: try 0.1 instead of 0.2 4. Increase window size: try 60 days instead of 50 5. Add a third LSTM layer with return_sequences=True Document every attempt and the resulting RMSE in a table.

Step 6 — Build the FastAPI Backend

Build and test one endpoint at a time. Never move to the next until the current one returns correct results.

37. **Load models and scaler at startup** (not inside the endpoint function — loading .h5 files is slow and must happen only once):

Model loading at startup:

```
from fastapi import FastAPI import joblib from tensorflow.keras.models import load_model
app = FastAPI() baseline = joblib.load('models/model_baseline.pkl') lstm =
load_model('models/model_lstm.h5') scaler = joblib.load('models/scaler.pkl')
```

38. Build GET /predict?ticker=AAPL: read CSV, take last 50 rows, scale, predict with both models, inverse-transform, return JSON
39. Build GET /history?ticker=AAPL: return last 90 days of raw closing prices as a list of {date, price} objects
40. Build GET /metrics?ticker=AAPL: return pre-calculated MSE and RMSE scores stored as a dictionary
41. Build GET /tickers: return the hard-coded list of 5 supported tickers
42. Add CORS middleware (see Section 2.4)
43. Test every endpoint at http://localhost:8000/docs before building the frontend

Step 7 — Build the React Frontend

Build in layers. Never attempt the full dashboard in one go.

44. **Start:** text input for ticker + button + raw JSON displayed in a pre tag. Confirm the API connection works end-to-end.
45. **Historical chart:** call /history and render a Chart.js line chart of the past 90 days.
46. **Add predictions:** overlay two marker points at day 91 — one in green (LSTM) and one in orange (Baseline).
47. **Metrics panel:** call /metrics and display RMSE badges in matching colours below the chart.
48. **Disclaimer:** add a fixed banner at the very top: "For educational purposes only. Not financial advice." This cannot be removed.

The most common React deployment mistake:

Never hardcode http://localhost:8000 in API calls. Use: const API_URL = process.env.REACT_APP_API_URL. Set this in a .env file locally and in Render's environment variables for production. Forgetting this means the deployed app talks to nothing.

Step 8 — Testing and Evaluation

Two types of testing are required:

Model accuracy testing

For each of the 5 tickers, record MSE and RMSE for both models on the test set. Present results in a table. Confirm LSTM achieves ≥ 10% RMSE improvement. If not, document the shortfall and the tuning attempts honestly.

API and usability testing

Test each endpoint in Postman. Try invalid tickers — confirm the API returns a 422 error with a clear message. Ask 3–5 people to use the dashboard. Record what confused them and what you changed as a result.

Step 9 — Deploy to Render

49. **FastAPI backend:** New Web Service on Render, connect GitHub repo, root directory: backend/, start command: uvicorn main:app --host 0.0.0.0 --port $PORT, runtime Python 3
50. **React frontend:** New Static Site on Render, connect same repo, root directory: frontend/, build command: npm run build, publish directory: build
51. **Set environment variable:** Add REACT_APP_API_URL=https://your-api-name.onrender.com in Render's environment settings for the Static Site
52. Test the live deployment with at least two tickers before submission

5. Risks and How to Handle Them

Risk	Level	What to do
LSTM fails to hit 10% RMSE improvement	HIGH	Tune: more units, more epochs, less dropout, different window size. Document every attempt.
Time-series data shuffled accidentally	HIGH	Never use train_test_split with shuffle=True. Always split by index position (chronological).
Scaler not saved alongside models	HIGH	Save scaler.pkl immediately after fitting. Without it, production predictions will be wrong.
Model overfitting to training data	MEDIUM	Use Dropout(0.2) in LSTM layers. Monitor val_loss — if it rises while train_loss falls, overfitting is happening. Use EarlyStopping.
API rate limits hit during development	MEDIUM	Cache all historical data as CSV files. Only call yfinance for the most recent single day's price.
CORS blocking React → FastAPI calls	MEDIUM	Add CORSMiddleware on day one. Test locally before deployment.
REACT_APP_API_URL pointing to localhost in production	MEDIUM	Always use the environment variable. Never hardcode localhost in React.
Models too large to deploy on free tier	LOW	Reduce LSTM units (32/16) if .h5 file exceeds Render's free tier limits. Lighter models are still valid.
Users treat predictions as real financial advice	LOW	Prominent disclaimer on every page. No buy/sell language anywhere in the UI.

6. Questions to Ask Before Handing Over

Technical Questions

53. Have you used pandas before, specifically with time-series or financial data?
54. Have you built or trained a neural network before, even a simple one?
55. Do you understand what an LSTM is and how it differs from a standard RNN?
56. Have you used TensorFlow or Keras before, or are you willing to learn from scratch?
57. Have you built a REST API before using FastAPI, Flask, or similar?
58. Have you used React.js before?
59. Are you comfortable reading API documentation and working with JSON responses?
60. Do you use Git and GitHub regularly?
61. Have you deployed a web application to the cloud before?

Logistics Questions

62. Do you have a computer with Python 3.10+, Node.js, and enough RAM for LSTM training (8GB+ recommended)?
63. Can you show concrete progress at the end of each key week (Weeks 3, 4, 5, 6, 7, 8, 9, 10, 12)?

64. Are you taking other heavy modules this semester that might compete for your time?
65. Do you have a free account on Render or Heroku for deployment?

7. Quick Reference

Item	Detail
Project title	Real-Time Stock Market Forecaster
Student	Mehmet Tanil Kaplan — T0429362 — NTU — Supervisor: Williams Magdalena
Frontend	React.js + Chart.js + Axios
Backend	FastAPI (Python) + uvicorn + joblib
Baseline model	Linear Regression or Random Forest — saved as model_baseline.pkl
Deep learning model	LSTM (TensorFlow / Keras) — saved as model_lstm.h5
Data sources	Yahoo Finance (yfinance) or Alpha Vantage API — OHLCV daily data
Stock tickers	At least 5: AAPL, TSLA, MSFT, GOOGL, AMZN recommended
Window size	50 days input → predict day 51 closing price
Data split	70% train / 15% validation / 15% test — chronological, never shuffled
Accuracy target	LSTM must achieve ≥ 10% lower RMSE than baseline across all 5 tickers
API response target	Under 1 second per /predict call
Deployment	Render or Heroku — backend as Web Service, frontend as Static Site
Timeline	12 weeks with weekly milestones
Ethical requirement	"For educational purposes only. Not financial advice." on every page.

End of Project 2 Handover & Build Guide

Mehmet Tanil Kaplan • T0429362 • Nottingham Trent University • 2025–2026